CS2810

Object Oriented Programming Lab

Module: File I/O & STL Sequential Containers

Time Limit: 90 Minutes — **Max Marks:** 50

Assignment Brief

Low-Latency Software for Trading

Congratulations! You have just been hired by **IMC Trading's Mumbai office** as a Core C++ Developer. Your starting annual base package is **1.2 Crores Rupees**, and expectations are high.

The CTO has assigned you to the Core Engineering team for their proprietary High-Frequency Trading (HFT) platform. Your first mission is to build a compact analytics + execution utility that can:

- analyze historical order files,
- answer user-level quantity queries, and
- execute incoming orders into transaction logs.

You must implement all parts using C++ with STL sequential containers and robust file I/O.

Part 1: Market Snapshot (Ticker Order Counts) [Marks: 20]

Read a CSV file and report the total number of orders per company ticker.

CSV Format

Each CSV row has the fields (comma-separated):

`Type,UserName,CompanyTicker,Quantities,Price`

where `Type` is `BUY` or `SELL`.

Input

1. A line containing the part indicator: `P1`
2. One line: the complete path to the input CSV file (including filename)

Console Output

Print lines in the form:

`<CompanyTicker> <TotalNumberOfOrders>`

for every ticker present in the file, sorted lexicographically by ticker.

Examples

Example 1: Mumbai Tech Traders

User Input (stdin):

```
P1
./data/trading_orders.csv
```

Input CSV (trading_orders.csv):

```
BUY,Rajesh,INFY,100,2450.50
SELL,Priya,TCS,50,3100.00
BUY,Arjun,INFY,200,2455.00
BUY,Divya,WIPRO,75,450.00
SELL,Vikram,INFY,100,2460.00
```

Console Output:

```
INFY 3
TCS 1
WIPRO 1
```

Example 2: Silicon Valley & India Mix

User Input (stdin):

```
P1
./market/global_orders.csv
```

Input CSV (global_orders.csv):

```
BUY,Aisha,GOOGL,10,180.00
BUY,Karan,AMZN,20,190.00
SELL,Meera,GOOGL,5,175.00
BUY,Nikhil,HCL,100,1250.00
SELL,Priya,AMZN,15,185.00
SELL,Arjun,INFY,50,2500.00
```

Console Output:

```
AMZN 2
GOOGL 2
HCL 1
INFY 1
```

Part 2: Trader Exposure (User Quantity Aggregation) [Marks: 20]

Read the same CSV format and answer queries for user totals.

Input

1. A line containing the part indicator: `P2`
2. One line: the complete path to the input CSV file (including filename)
3. Then one or more lines containing a `UserName` per line (the number of query lines is not fixed). Process all provided user lines until EOF.

Console Output

For each queried user name, print a line:

`<UserName> <TotalQuantities>`

where `TotalQuantities` is the sum of the `Quantities` field for all orders (both BUY and SELL) placed by that user across all tickers. If the user has no orders, print zero.

Examples

Example 1: Trader Risk Analysis

Input CSV (portfolio.csv) with queries:

```
P2
./data/portfolio.csv
Rajesh
Priya
UnknownTrader
```

CSV File Content (portfolio.csv):

```
BUY,Rajesh,INFY,100,2450.50
SELL,Rajesh,TCS,50,3100.00
BUY,Priya,WIPRO,200,450.00
SELL,Priya,INFY,75,2455.00
BUY,Vikram,GOOGL,10,180.00
```

Console Output:

```
Rajesh 150
Priya 275
UnknownTrader 0
```

Example 2: Multi-Ticker Exposure

Input CSV (multi_trader.csv) with queries:

```
P2
./market/multi_trader.csv
Arjun
Divya
```

CSV File Content (multi_trader.csv):

```
BUY,Arjun,INFY,150,2500.00
BUY,Arjun,HCL,100,1200.00
SELL,Arjun,AMZN,50,185.00
BUY,Divya,GOOGL,20,175.00
SELL,Divya,TCS,100,3050.00
BUY,Divya,WIPRO,80,450.00
```

Console Output:

```
Arjun 300
Divya 200
```

Part 3: Live Matching Engine with Transaction Logging [Marks: 10]

Accept live orders from input, execute matches, and write executed transactions to a CSV file.

CSV Input/Output Formats

Input order rows use the same CSV-style fields but provided via standard input lines as space-separated values:

`(BUY/SELL) (UserName) (CompanyTicker) (Quantities) (Price)`

Output transaction CSV must use the header and format:

`Ticker,Seller,Buyer,Qty,Price,Time`

Note: All price values in the output CSV must be formatted to exactly 2 decimal places using `fixed << setprecision(2)`.

Input

1. A line containing the part indicator: `P3`
2. One line: the **filename** of the output CSV file (e.g., `executed_trades.csv`). The full output path must be constructed as:

   `./actual_output/Q1/CSV/<YOUR_ROLL_NUMBER>/<filename>`

   Replace `<YOUR_ROLL_NUMBER>` with your own roll number.
3. Then one or more lines of orders (space-separated, not comma-separated). Process all provided order lines until an empty line or EOF.

Execution Rules

Follow these rules when matching orders:

- A transaction executes when a buy and sell order satisfy `Sell.Price` $\leq$ `Buy.Price`.
- Execution price is the seller's price.
- Executed quantity is the minimum of remaining buy and sell quantities; leftovers remain in the order book.
- **Matching Priority & Best Price Logic:**
  - An incoming `BUY` order matches against existing `SELL` orders for the same ticker, prioritizing the **lowest** SELL price first. If multiple SELL orders have the same price, use **time-priority**: match with the oldest (earliest entered) SELL order.
  - An incoming `SELL` order matches against existing `BUY` orders for the same ticker, prioritizing the **highest** BUY price first. If multiple BUY orders have the same price, use **time-priority**: match with the oldest (earliest entered) BUY order.
  - Example: If market has SELL orders at prices [2445, 2445, 2450] for INFY and a BUY arrives at 2450, it matches the first SELL at 2445 (lowest price), then the second SELL at 2445 (time-tie), then third at 2450.
- Time starts at 0 and increases by 1 for each executed transaction.

Console Output

No console output is required for `P3`.

Files and Notes

Write the executed transactions to the constructed output CSV path (`./actual_output/Q1/CSV/<YOUR_ROLL_NUMB` using the header above. Ensure the directory exists before writing. You do not need to read any pre-existing file before writing.

Examples

Example 1: Quick Execution with INFY

Input Commands:

```
P3
executed_trades.csv
BUY Rajesh INFY 100 2450.00
SELL Priya INFY 60 2440.00
BUY Arjun TCS 50 3100.00
SELL Divya TCS 50 3090.00
```

Generated Output CSV (./actual_output/Q1/CSV/<ROLL>/executed_trades.csv):

```
Ticker,Seller,Buyer,Qty,Price,Time
INFY,Priya,Rajesh,60,2440.00,0
TCS,Divya,Arjun,50,3090.00,1
```

Example 2: Partial Fills with Multiple Tickers

Input Commands:

```
P3
livematch.csv
BUY Vikram GOOGL 50 180.00
SELL Meera GOOGL 30 175.00
BUY Aisha WIPRO 100 450.00
SELL Karan WIPRO 150 445.00
BUY Nikhil GOOGL 30 182.00
```

Generated Output CSV (./actual_output/Q1/CSV/<ROLL>/livematch.csv):

```
Ticker,Seller,Buyer,Qty,Price,Time
GOOGL,Meera,Vikram,30,175.00,0
WIPRO,Karan,Aisha,100,445.00,1
```

(*Note:* After these trades, the order book contains: Vikram's remaining BUY order for 20 GOOGL shares @ 180.00, Karan's remaining SELL order for 50 WIPRO shares @ 445.00, and Nikhil's BUY order for 30 GOOGL shares @ 182.00. No match occurs between Nikhil's BUY and Vikram's BUY as both are buy-side orders.)

General Implementation Hints

- Use STL container `std::vector`
- Use `std::ifstream`/`std::ofstream` for file I/O. Read filenames from input; do not hardcode.
- Parse CSV lines with `std::getline` and `std::stringstream` when needed.